Solving TAPF via Poisson Potential Fields and Heuristic Word Reduction in Permutation Groups

Boxiang (William) Fu
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213
boxiangf@andrew.cmu.edu

F.D.C. Plato
School of Feline Studies
Cat Meowie University
Purrsburgh, PA 15213
(no email)

***Abstract*—We present a new formulation of target allocation and path finding (TAPF) that recasts the problem as a constrained word problem in a permutation group generated by graph-edge transpositions. Based on this view, we propose a two-level solver that combines global Poisson potential-field guidance with local greedy word reduction. The high-level planner induces a dynamic goal allocation without fixing assignments in advance, while the low-level planner constructs feasible local transposition sequences through greedy pebble-motion moves, recursive blocker displacement, and backtracking. Experiments on benchmarks show that the method solves most large benchmark instances with up to 200 agents at low runtime and search tree nodes expanded. The framework also supports execution-time delay handling by rerunning the word-to-path conversion step without full replanning. Overall, the results suggest that combining potential-field guidance with group-theoretic local motion is a promising alternative to conventional graph-search-based TAPF solvers.**

***Index Terms*—multi-agent path finding, target allocation and path finding, permutation groups, group actions, Poisson potential fields, pebble motion, multi-robot planning**

I. Introduction

Multi-Agent Path Finding (MAPF) and Task Allocation and Path Finding (TAPF) are central problems in multi-robot coordination. Such problems arise in scenarios such as warehouse coordination and drone swarm control [9], [21]. In MAPF, each agent is assigned a fixed start and goal, and the objective is to compute collision-free paths. In TAPF, the assignment between agents and goals must also be determined. Classical approaches typically formulate these problems as graph-search problems over joint or constrained configuration spaces. While effective, such formulations are often slow and does not always exploit the problem's structural properties and symmetries.

In this work, we consider an alternative formulation that recasts TAPF as a constrained word problem in a permutation group generated by graph edge transpositions. Each local swap along an edge acts as a generator, and a solution is a feasible non-colliding sequence of generators that transforms the start arrangement into the goal arrangement.

Based on this formulation, we propose a two-level solver that combines global Poisson potential-field guidance with local greedy word reduction. The high-level procedure uses a Poisson field over free space to induce a dynamic agent-goal allocation, while the low-level procedure constructs a feasible sequence of local transpositions by greedily moving agents along preferred directions and recursively displacing blockers when necessary. A search tree and backtracking mechanism is also used when the induced allocation becomes locally unsolvable.

We demonstrate our method by benchmarking on Moving AI Lab maps with increasing numbers of agents [16]. The results show that the proposed solver achieves fast runtime across multiple environments. suggesting that combining Poisson-field guidance with group-theoretic pebble-moving local motion is a promising alternative to conventional graph-search-based TAPF formulations.

II. Related Work

A large body of work studies MAPF and TAPF as graph-search problems over joint or constrained configuration spaces. Representative MAPF methods include Joint-State A*, CBS, PBS, PIBT, and LaCAM [8], [10], [11], [14], [15]. Related TAPF methods include CBM, CBS-TA, and ITA-CBS [5], [17], [18]. In contrast, our work is motivated by a group-theoretic view in which graph edges act as adjacent transposition generators and planning is recast as a constrained word problem in the automorphism group generated by those swaps. This connects naturally to work on group actions, symmetry groups, pebble motion, and token swapping in the mathematical literature [4], [6], [7], [13].

Our high-level planner also relates to potential-field-based planning. Previous work on path planning via navigation functions has been explored extensively in the literature. Most notably, [2] first proposed the use of Laplace's equation for path planning. Extensions of this approach include solving for heat transfer functions, Hamiltonians, and artificial potential fields [1], [19], [20]. This was further extended in [12] to lifelong MAPF problems.

III. Preliminaries

A. Groups & Group Actions

We briefly review the group-theoretic notions used in this work.

Definition 1 (Group)**.** *A* group $(G, *)$ *is a set G together with a binary operation*

$$* : G \times G \to G$$

such that:

1) ***Associativity:***

$$g * (h * k) = (g * h) * k \qquad \forall g, h, k \in G.$$

2) ***Identity:*** *There exists an element $e \in G$ such that*

$$g * e = e * g = g \qquad \forall g \in G.$$

3) ***Inverse:*** *For every $g \in G$, there exists an element $g^{-1} \in G$ such that*

$$g * g^{-1} = g^{-1} * g = e.$$

The element e is called the identity *of the group, and g^{-1} is called the* inverse *of g.*

Definition 2 (Group Action)**.** *Let G be a group and let X be a set. A* group action *of G on X is a function*

$$G \times X \to X, \qquad (g, x) \mapsto g \cdot x$$

such that:

1) ***Identity:***

$$e \cdot x = x \qquad \forall x \in X.$$

2) ***Group Operation:***

$$(gh) \cdot x = g \cdot (h \cdot x) \qquad \forall g, h \in G,\ x \in X.$$

When these conditions hold, we say that G acts on *X, and write*

$$G \curvearrowright X.$$

Definition 3 (Automorphism Group)**.** *Let V be a set. The* automorphism group *on V, denoted $\mathrm{Aut}(V)$, is the set of all bijections from V to itself:*

$$\mathrm{Aut}(V) = \{\phi : V \to V \mid \phi \text{ is bijective}\}.$$

Under composition of functions, $\mathrm{Aut}(V)$ forms a group.

Definition 4 (Group Generators)**.** *Let H be a group and let $S \subseteq H$. We say that S* generates *H if every element $h \in H$ can be written as a finite product of elements of S and their inverses. In this case, we write*

$$H = \langle S \rangle.$$

The elements of S are called generators *of H.*

Definition 5 (Constrained Word Problem)**.** *Let $H = \langle S \rangle$ be a group generated by a set S, and let $\sigma \in H$ be a target permutation. The* constrained word problem *is the problem of finding a finite sequence of generators*

$$s_1, s_2, \ldots, s_m \in S$$

such that

$$s_1 s_2 \cdots s_m = \sigma,$$

while also satisfying feasibility constraints. In the multi-agent setting, these constraints encode collision avoidance.

B. Poisson's Equation

We briefly review the differential operators and boundary value problem used in this work.

Definition 6 (Laplacian)**.** *Let $\phi : \Omega \to \mathbb{R}$ be a twice continuously differentiable function on a open domain $\Omega \subset \mathbb{R}^n$. The* Laplacian *of ϕ is defined by*

$$\Delta\phi \;=\; \sum_{i=1}^{n} \frac{\partial^2 \phi}{\partial x_i^2}.$$

Definition 7 (Neumann Boundary Condition)**.** *Let $\partial\Omega$ denote the boundary of Ω, and let n be the outward unit normal vector on $\partial\Omega$. The* Neumann boundary condition *prescribes the normal derivative of ϕ on the boundary:*

$$\frac{\partial \phi}{\partial n} = \nabla\phi \cdot n \qquad \text{on } \partial\Omega.$$

In particular, the homogeneous Neumann boundary condition is

$$\frac{\partial \phi}{\partial n} = 0 \qquad \text{on } \partial\Omega.$$

Definition 8 (Poisson Problem with Homogeneous Neumann Boundary Conditions)**.** *Let $\Omega \subset \mathbb{R}^n$ be a bounded domain with sufficiently regular boundary $\partial\Omega$, and let $\rho : \Omega \to \mathbb{R}$ be the charge density in the domain. The* Poisson problem with homogeneous Neumann boundary conditions *is to find*

$$\phi \in C^2(\Omega) \cap C^1(\overline{\Omega})$$

such that

$$\begin{cases} \Delta\phi = \rho, & \text{in } \Omega, \\ \dfrac{\partial \phi}{\partial n} = 0, & \text{on } \partial\Omega. \end{cases}$$

When a solution exists, it is unique up to an additive constant.

C. Upper Confidence Bound Trees

We briefly review the search tree and Upper Confidence Bound (UCB) rule used in this work.

Definition 9 (Search Tree)**.** *A* search tree *is a rooted tree $\mathcal{T} = (\mathcal{V}, \mathcal{E}, r)$, where each node represents a state and each edge represents a transition between states. The root r is the initial state, and every node is reachable from r by a unique path.*

Definition 10 (Upper Confidence Bound)**.** *Let v be a node in a search tree with parent $p(v)$. The* Upper Confidence Bound *score of v is defined by*

$$\mathrm{UCB}(v) = V(v) + c\sqrt{\frac{\log\big(N(p(v)) + 1\big)}{N(v)}},$$

where $V(v)$ is the value of node v, $N(v)$ is its visit count, $N(p(v))$ is the visit count of its parent, and $c > 0$ is an exploration constant. The first term favors nodes with high estimated value, while the second term favors nodes that have been explored less frequently.

IV. Problem Definition

Given a workspace graph (map) $G = (V, E)$, we associate each edge $(u, v) \in E$ the transposition $(u\ v)$, which swaps the occupancy of the two adjacent vertices while fixing all other vertices. The group generated by these graph-edge transpositions is

$$H = \langle (u\ v) \mid (u, v) \in E \rangle.$$

Thus, H is the permutation group obtained by composing local adjacent swaps induced by the graph structure. For a complete graph, this corresponds to the automorphism (symmetric) group on V that contains all the bijections from V to itself. In other words, $H = \mathrm{Aut}(V) = \mathrm{Sym}(V)$.

Under this perspective, the MAPF problem can be recast as a constrained word problem in H. Given a set of agents $A = \{1, \ldots, n\}$, start locations $S = (s_1, \ldots, s_n)$, and goal locations $G = (g_1, \ldots, g_n)$, the objective is to find a feasible sequence of generators in H whose composition transforms the start arrangement into the goal arrangement subject to vertex constraints (two adjacent transpositions in the same timestep cannot be transitioning into the same vertex) and edge constraints (two adjacent transpositions in the same timestep cannot be copies). This problem can be solved efficiently using pebble-motion algorithms [7].

We extend this formulation from MAPF to anonymous TAPF by removing any fixed initial assignment between agents and goals. Instead, the allocation is induced naturally by allowing agents (particles) to follow the Principle of Least Action [3]. The agents are modeled as positively charged sources, the goal locations as negatively charged sinks, and a Poisson potential field is solved over the free space with Neumann boundary conditions at obstacles and map boundaries. The resulting field provides a global downhill direction that guides local pebble motions.

V. Methodology

A. Low-Level Pebble Motion Step

The low-level solver uses a greedy procedure to solve the constrained pebble-motion problem. At each iteration, a single active agent is selected via the agent selection algorithm and its legal neighboring cells are enumerated. These legal candidates are ranked using a goal-directed heuristic (potential field or Dijkstra search). The most preferred move is the one with the greatest gradient descent.

If the highest-ranked move is unoccupied, the agent is moved directly. If the preferred cell is occupied by another agent, the solver attempts to recursively displace the blocking chain to free that cell. If a blocking chain can be successfully pushed, the original agent then executes its preferred move. If recursive displacement fails, the selected agent falls back to less preferred alternative moves. To reduce oscillatory behavior, these fallback moves are restricted to non-backtracking actions. If no available move exists, the local step returns failure, and the high-level planner would backtrack to a more promising state. The pseudo-code for the low-level pebble motion algorithm is shown in Algorithm 1.

Algorithm 1 Low-Level Pebble Motion Step

```
L: current agent locations,  T: transposition sequence
Get legal moves N of agent i
Rank N and let best move be v
if v is free then
    Move i to v, update L, append (i, v) to T
else
    Let j be the agent blocking v
    if blocking agents can be recursively moved to free v then
        Recursively move blocking agents, update L, append transpositions to T
        Move i to v, update L, append (i, v) to T
    else if i has a valid non-backtracking move u then
        Move i to u, update L, append (i, u) to T
    else
        return failure
    end if
end if
```

Algorithm 2 Poisson Solver

```
Pad map with obstacle boundary
Remove non-connected free-space components
Flatten grid to indices k = r · w + c
for each free 4-neighbor pair (p, q) do
    Set A_pq ← −1
end for
for each cell p do
    if p is free and not fixed then
        Set A_pp ← deg(p)
    else
        Set A_pp ← 1
    end if
end for
Set b_p ← +α for starts, b_p ← −α for goals
Solve Aφ = b
Reshape, zero obstacle cells, and crop boundary
return φ
```

B. High-Level Potential Field Planning

Unlike traditional high-level TAPF planners, our method removes any fixed initial agent-goal assignment and instead induces the allocation naturally by allowing agents (particles) to follow the Principle of Least Action [3]. Motivated by electrostatic theory, the current active agent locations are modeled as positively charged sources, and the active goal locations as negatively charged sinks. Poisson's equation is then solved over this domain with homogeneous Neumann boundary conditions at obstacles and map boundaries. The resulting potential field ϕ bias agents toward the remaining goals without explicitly committing to an assignment in advance. The pseudo-code for computing the potential field is shown in Algorithm 2.

Planning then alternates between the high-level planner and

Algorithm 3 High-Level Potential Field Planning

```
Initialize locations L ← S, moves T ← ∅, active agents A ← {1, ..., n}
Precompute potential field φ from active starts to active goals
while L ≠ G and time not exceeded do
    Choose agent i from A
    if L[i] ∈ G then
        Mark i inactive, update active goals, recompute φ
        Commit current moves T to the search tree
        continue
    end if
    if PebbleMotion(i, L, T, φ) succeeds then
        Update L and T
    else
        if backtrack by UCB succeeds then
            Restore previous state
            continue
        else
            break
        end if
    end if
end while
return Convert T to agent paths
```

the local pebble-motion solver. At each iteration, a currently active agent is selected, and the PEBBLEMOTION algorithm attempts to advance that agent using the current potential field. If the agent reaches a goal, it is marked inactive, the reached goal is removed from the active goal set, and the potential field is recomputed over the remaining subproblem. The solver also commits the current transposition sequence to a high-level search tree node. This allows the high-level planner to backtrack to more promising configurations should this branch reach a deadlock. The pseudo-code for the high-level planner is depicted in Algorithm 3.

C. Backtracking

When the local greedy planner fails to make progress, the high-level planner backtracks in the search tree using an Upper Confidence Bound (UCB) rule. For a node v with parent $p(v)$, the score is computed as

$$\mathrm{UCB}(v) = V(v) + c\sqrt{\frac{\log\big(N(p(v))+1\big)}{N(v)}},$$

where $V(v)$ is the stored heuristic value of node v, $N(v)$ is its visit count, $N(p(v))$ is the visit count of its parent, and $c > 0$ is an exploration constant. We calculate a node's value as

$$V(v) = \frac{\big(n - |A(v)|\big)\,(2s) + \sum_{i\in A(v)} \big\|-\nabla\phi\big(L_i(v)\big)\big\|_2}{N(v)},$$

where n is the total number of agents, $A(v)$ is the set of active agents at node v, $L_i(v)$ is the current location of agent i at that node, and s is the source potential strength. This formulation encodes the estimated reward for completed agents with a measure of the remaining downhill potential available to active agents. The backtrack function evaluates previously created nodes using $\mathrm{UCB}(v)$ and restores the state of the node with maximum score, and resumes planning from that earlier node.

D. Agent Selection

At each high-level iteration, the solver selects one active agent to expand. Each active agent i is assigned a priority score

$$\mathrm{score}(i) = w_i\Big(1 + \beta\, b_i\Big),$$

where

$$w_i = t - \ell_i$$

is the number of iterations since agent i was last selected, b_i is the number of times agent i has triggered backtracking, ℓ_i is the last-picked iteration of agent i, t is the current selection iteration, and $\beta > 0$ is a fixed backtrack ratio. The selected agent is the active agent with maximum $\mathrm{score}(i)$, with ties broken in favor of the agent that has waited longer.

E. Word-to-Path Conversion

After the solver constructs a feasible word in graph-edge transpositions, it must be converted into agent paths. The conversion procedure scans the transpositions in chronological order and groups them into timesteps. A new timestep is started whenever adding the next transposition would introduce either a repeated agent or a vertex conflict within the current timestep. The constrained word problem solution guarantees that a feasible agent path must exist – with the worst case scenario being that each transposition is allocated its own timestep, so that the agent moves one at a time.

The pseudo-code for this conversion is shown in Algorithm 4. Waiting actions are inserted for all agents that do not have their own word in the partitioned timestep.

VI. EXPERIMENTS

We evaluate our TAPF solver on the course homework instances and three grid maps from Moving AI Lab: `maze-32-32-2`, `random-32-32-10`, and `room-64-64-8`. For each Moving AI Lab map, agent start and goal locations are generated randomly in free space, and the number of agents is increased from 10 to 200 in increments of 10. The initial map layout, potential field, and gradient field for the three Moving AI Lab maps are shown in Figures 1, 2, and 3. For every instance, we record whether the solver finds a solution within the time limit, the resulting sum of costs, the runtime, and the number of generated high-level tree nodes. We summarize the obtained results in Tables I, II, III, and IV.

Algorithm 4 Word-to-Path Conversion

Require: Transposition queue T, number of agents n, start locations S

Ensure: Paths $\Pi = \{\pi_1, \ldots, \pi_n\}$

Initialize timestep queue $\mathcal{T} \leftarrow \emptyset$, current timestep $\tau \leftarrow \emptyset$, used-agent set $A \leftarrow \emptyset$, used-destination set $D \leftarrow \emptyset$
while $T \neq \emptyset$ **do**
  Pop $(i, (v_{\text{end}}, v_{\text{start}}))$ from T
  if $i \in A$ **or** $v_{\text{end}} \in D$ **then**
    Push τ into $\mathcal{T}$ and reset τ, A, D
  end if
  Add $(i, v_{\text{end}}, v_{\text{start}})$ to τ
  $A \leftarrow A \cup \{i\}, \quad D \leftarrow D \cup \{v_{\text{end}}\}$
end while
Initialize each path $\pi_i \leftarrow [s_i]$
for each timestep τ in chronological order **do**
  Let M be the set of agents moving in τ
  for each $(i, v_{\text{end}}, v_{\text{start}}) \in \tau$ **do**
    Append v_{end} to π_i
  end for
  for each agent $i \notin M$ **do**
    Append the previous location of π_i to π_i
  end for
end for
return Π

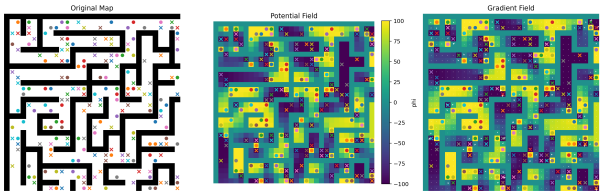

Fig. 1. Initial Layout on `maze-32-32-2` (100 agents)

VII. Evaluation

The experimental results shows that our TAPF solver is effective across both the course homework instances and the Moving AI Lab benchmarks. All course homework instances were solved in less than 0.022 s of CPU time. With backtracking, the solver is also able to effectively solve scenarios with a large number of agents (`test_16` with 25 agents) and single-agent corridors (`test_20` and `test_21`).

Across the three Moving AI Lab benchmarks, the solver is able to efficiently solve most instances up to 200 agents. Runtime stays below 3 s for most instances, and the number of nodes expanded remains within 1000. Two failure cases occurred in the `room-64-64-8` map for the 100-agent and 170-agent cases. Both failure cases were caused by two agents simultaneously claiming ownership of a single goal location, thereby causing a livelock where each agent claims ownership of the goal for one timestep before switching ownership. A potential fix could be to also backtrack when a livelock is detected, which could be worthwhile to explore in future works.

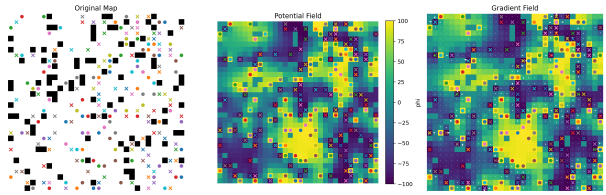

Fig. 2. Initial Layout on `random-32-32-10` (100 agents)

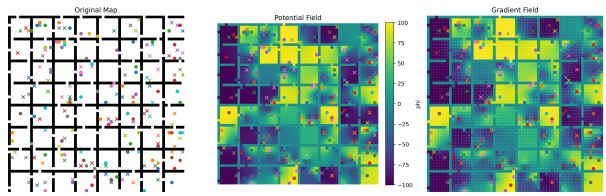

Fig. 3. Initial Layout on `room-64-64-8` (110 agents)

An additional advantage of the proposed framework is that execution can naturally accommodate agent delays without requiring the full TAPF problem to be replanned. Since the solver first produces a feasible sequence of pebble-motion transpositions and only afterward converts that sequence into agent paths, execution-time delays can be handled by re-running the word-to-path conversion step. In particular, if one agent is delayed, the executor can simply regroup the unexecuted pebble-motion permutation sequence into a new set of non-conflicting timestep blocks. This saves compute as we forgo having to recompute an entirely new plan.

VIII. Future Work

Several directions for extending this work could be taken beyond the scope of this project. The first action to take is to incorporate livelock detection into the backtracking mechanism. This would allow the planner to solve failure cases where two agents simultaneously claim ownership of a single goal location. A second direction is to study the theoretical properties of the methodology. The current method is neither complete nor optimal. With reasonable effort, the method could potentially be made complete by altering the way the high-level search tree expands its branches. Finally, the method could be generalized from anonymous single-team TAPF to multi-team TAPF with heterogeneous agent and goal types. This would require storing different "flavors" of the potential field for each team of agents. Agents would only interact with the potential field of their own "flavor" class.

IX. Conclusion

In this paper, we presented a new perspective on anonymous TAPF by recasting it as a constrained word problem in a permutation group generated by graph-edge transpositions. Our proposed two-level solver combines global Poisson potential-field guidance with local greedy word reduction. Empirically, the method performs strongly on both the course homework instances and the Moving AI Lab benchmarks. It is able to solve most large benchmark instances of up to 200

TABLE I
RESULTS ON COURSE HOMEWORK INSTANCES

Instance	Cost	CPU Time (s)	Node Count
test_0	40	0.001	6
test_1	78	0.002	9
test_2	24	0.002	8
test_3	30	0.002	7
test_4	36	0.002	9
test_5	49	0.002	8
test_6	63	0.002	9
test_7	63	0.003	10
test_8	49	0.002	9
test_9	70	0.003	10
test_10	70	0.003	9
test_11	49	0.002	8
test_12	63	0.003	12
test_13	35	0.002	8
test_14	63	0.002	9
test_15	276	0.013	19
test_16	625	0.022	30
test_17	348	0.009	18
test_18	54	0.002	8
test_19	330	0.011	18
test_20	252	0.006	21
test_21	182	0.004	17

TABLE II
RESULTS ON MAZE-32-32-2

Agents	Cost	CPU Time (s)	Node Count
10	1190	0.018	13
20	3360	0.056	37
30	5310	0.089	59
40	5760	0.163	105
50	7700	0.196	107
60	10260	0.173	110
70	11130	0.219	142
80	10800	0.326	204
90	15030	0.351	219
100	19600	0.439	248
110	17050	0.392	247
120	28080	0.578	364
130	26520	0.603	359
140	38920	0.846	510
150	41250	0.893	526
160	33120	0.719	441
170	70550	1.426	898
180	63720	1.315	822
190	43700	0.961	555
200	37000	0.800	465

TABLE III
RESULTS ON RANDOM-32-32-10

Agents	Cost	CPU Time (s)	Node Count
10	420	0.024	11
20	1140	0.059	26
30	1560	0.086	37
40	3120	0.136	57
50	1700	0.154	64
60	3060	0.181	74
70	2800	0.213	87
80	4240	0.271	111
90	7560	0.387	145
100	8500	0.402	145
110	7260	0.392	152
120	9600	0.518	206
130	8580	0.545	217
140	11900	0.596	236
150	12300	0.666	262
160	12000	0.651	258
170	12750	0.722	282
180	18000	0.834	317
190	14250	0.801	305
200	16800	0.879	337

TABLE IV
RESULTS ON ROOM-64-64-8

Agents	Cost	CPU Time (s)	Node Count
10	1140	0.064	11
20	4280	0.241	32
30	5130	0.332	43
40	10920	0.923	116
50	8900	0.532	69
60	15360	1.541	200
70	19530	0.932	116
80	22320	0.975	121
90	19260	0.980	122
100	-1	30.003	4483
110	41250	1.559	186
120	25080	1.445	173
130	36140	1.976	234
140	57260	2.732	330
150	35250	1.989	236
160	44480	2.347	281
170	-1	30.002	4100
180	32220	2.133	256
190	42560	2.294	267
200	71800	3.237	391

agents with low runtime and number of nodes expanded. The failure cases have been identified as livelocks and can be resolved by extending the backtracking method in future work. Furthermore, our method is also able to account for agent delays without full replanning by simply rerunning word-to-path conversion (Algorithm 4). Overall, these results suggest that combining potential-field guidance with group-theoretic local motion provides a promising and efficient alternative to conventional graph-search-based TAPF solvers.

CONTRIBUTION STATEMENT

This project is completed as part of the Final Course Project for CMU 16-891 Multi-Robot Planning and Coordination (Spring 2026). The problem formulation, algorithm design, implementation, experiments, and writing were carried out by Boxiang (William) Fu. F.D.C. Plato is a very cute kitty and has provided me invaluable emotional support throughout this project. The code repository for this project is available at https://github.com/williamfbx/Group-Action-MAPF.

REFERENCES

[1] D. Casagrande, G. Fenu, and F. A. Pellegrino. Hamiltonian path planning in constrained workspace. *Eur. J. Control*, 33:1–10, 2017.

[2] C. Connolly, J. Burns, and R. Weiss. Path planning using laplace's equation. In *Proceedings., IEEE International Conference on Robotics and Automation*, pages 2102–2106 vol.3, 1990.

[3] R. P. Feynman. The principle of least action in quantum mechanics. *Thesis, Princeton University*, 1942.

[4] A. Ghitza. Group theory and linear algebra 2021. Lecture notes, MAST20022, 2021. Version of Wed 27th Oct, 2021 at 10:48.

[5] W. Hönig, S. Kiesel, A. Tinka, J. W. Durham, and N. Ayanian. Conflict-based search with optimal task assignment. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '18, page 757–765, Richland, SC, 2018. International Foundation for Autonomous Agents and Multiagent Systems.

[6] James. *Symmetric Groups and Their Young Subgroups*, page 1–33. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 1984.

[7] D. Kornhauser, G. L. Miller, and P. Spirakis. Coordinating pebble motion on graphs, the diameter of permutation groups, and applications. pages 241–250, Florida, October 1984. IEEE.

[8] H. Ma, D. Harabor, P. J. Stuckey, J. Li, and S. Koenig. Searching with consistent prioritization for multi-agent path finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 7643–7650, 2019.

[9] R. Morris, C. S. Păsăreanu, K. S. Luckow, W. A. Malik, H. Ma, T. K. S. Kumar, and S. Koenig. Planning, scheduling and monitoring for airport surface operations. In *AAAI Workshop: Planning for Hybrid Systems*, 2016.

[10] K. Okumura. Lacam: Search-based algorithm for quick multi-agent pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11655–11662, 2023.

[11] K. Okumura, M. Machida, X. Défago, and Y. Tamura. Priority inheritance with backtracking for iterative multi-agent path finding. *Artificial Intelligence*, 310:103752, 2022.

[12] A. Pertzovsky, R. Stern, A. Felner, and R. Zivan. Enhancing lifelong multi-agent path-finding by using artificial potential fields, 2025.

[13] J. J. Rotman. *The theory of groups, an introduction*. Allyn and Bacon series in advanced mathematics. Allyn and Bacon, Boston, 1965.

[14] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015.

[15] T. S. Standley and R. E. Korf. Complete algorithms for cooperative pathfinding problems. In *Proceedings of the Twenty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, pages 668–673, 2011.

[16] R. Stern, N. Sturtevant, A. Felner, S. Koenig, H. Ma, T. Walker, J. Li, D. Atzmon, L. Cohen, T. K. S. Kumar, E. Boyarski, and R. Bartak. Multi-agent pathfinding: Definitions, variants, and benchmarks, 2019.

[17] Y. Tang, S. Koenig, and J. Li. Ita-ecbs: A bounded-suboptimal algorithm for combined target-assignment and path-finding problem. *Proceedings of the International Symposium on Combinatorial Search*, 17(1):134–142, Jun. 2024.

[18] Y. Tang, Z. Ren, J. Li, and K. Sycara. Solving multi-agent target assignment and path finding with a single constraint tree, 2023.

[19] Y. Wang and G. S. Chirikjian. A new potential field method for robot path planning. *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, 2:977–982 vol.2, 2000.

[20] Z. Wu, J. Dai, B. Jiang, and H. R. Karimi. Robot path planning based on artificial potential field with deterministic annealing. *ISA Transactions*, 138:74–87, 2023.

[21] P. R. Wurman, R. D'Andrea, and M. Mountz. Coordinating hundreds of cooperative, autonomous vehicles in warehouses. *AI Magazine*, 29(1):9, Mar. 2008.